Married Write - Spend Protocol and Tickets

A LIVE hop is ink, not a grant. Spend requires a short-lived ticket bound to one write fingerprint.

Source: gate/spend_protocol.py + gate/ticket.py

Law

> *LIVE hop != spend grant. Ticket is bound to a fingerprint of the write. Redeem must present the same write. Stale hop is museum.*

Study focus

- How fingerprint is computed (method + path + job + spend kind)
- Ticket TTL and single-use semantics
- Why this closes TOCTOU between hop and write

spend_protocol.py

```
"""Spend protocol  -  the scanner.

A LIVE hop is not a print. The print is a ticket bound to one write:

  POST /job/v1/jobs/{job_id}/bind-only

Fingerprint is SHA-256 over canonical JSON of method + path + job_id + spend_kind.
Redeem must present that same write. A ticket for bind-only cannot authorize
bind-and-issue. Missing write -> fail closed.

This is the object others implement (Cloudflare, Gosu is a different door).
Not a church. The only scanner that can print a YES on this spend.
"""
from __future__ import annotations

import hashlib
import json
from typing import Any

SPEC = "gate-spend-protocol-v1"
SPEND_KIND = "bind"
METHOD = "POST"
PATH_TEMPLATE = "/job/v1/jobs/{job_id}/bind-only"
MGA_PATH = "/v1/pas/mga-authority"
REASON_NOT_IN_PROTOCOL = "spend_write_not_in_protocol"
REASON_REQUIRED = "spend_write_required"
REASON_MISMATCH = "ticket_spend_mismatch"


def _canonical(obj: Any) -> str:
    return json.dumps(obj, sort_keys=True, separators=(",", ":"), ensure_ascii=False)


def normalize_job_id(job_id: str | None) -> str:
    return (job_id or "").strip()
```

```
def bind_only_path(job_id: str) -> str:
    jid = normalize_job_id(job_id) or "JOB_ID"
    return PATH_TEMPLATE.format(job_id=jid)


def normalize_path(path: str | None) -> str:
    p = (path or "").strip()
    if len(p) > 1 and p.endswith("/"):
        p = p[:-1]
    return p


def write(
    *,
    job_id: str,
    method: str | None = None,
    path: str | None = None,
    spend_kind: str | None = None,
) -> dict:
    jid = normalize_job_id(job_id)
    return {
        "method": (method or METHOD).strip().upper(),
        "path": normalize_path(path) or bind_only_path(jid),
        "job_id": jid,
        "spend_kind": (spend_kind or SPEND_KIND).strip(),
    }


def fingerprint(write_obj: dict | None) -> str | None:
    if not isinstance(write_obj, dict):
        return None
    body = {
        "job_id": write_obj.get("job_id") or "",
        "method": write_obj.get("method") or "",
        "path": write_obj.get("path") or "",
        "spend_kind": write_obj.get("spend_kind") or "",
    }
    if not body["job_id"] or not body["method"] or not body["path"] or not body["spend_kind"]:
        return None
    return hashlib.sha256(_canonical(body).encode("utf-8")).hexdigest()


def is_bind_only_path(path: str | None, job_id: str) -> bool:
    return normalize_path(path) == normalize_path(bind_only_path(job_id))


def _action_token(action: str | None) -> str:
    return (action or "").strip().lower().replace("_", "-")


def intended_policycenter(
    *,
    job_id: str | None,
    action: str | None = None,
    method: str | None = None,
    path: str | None = None,
```

```
    bind_path: str | None = None,
) -> dict | None:
    """Married PolicyCenter write, or None if they asked for a door this scanner does not print.""
  "
    jid = normalize_job_id(job_id)
    if not jid:
        return None
    meth = (method or METHOD).strip().upper()
    if meth != METHOD:
        return None
    p = normalize_path(path or bind_path)
    act = _action_token(action)
    married = write(job_id=jid)
    if p:
        return married if is_bind_only_path(p, jid) else None
    if act:
        if act in {"bind-only", "bind"}:
            return married
        return None
    return married


def intended_mga(*, job_id: str | None) -> dict | None:
    jid = normalize_job_id(job_id)
    if not jid:
        return None
    return write(job_id=jid, path=MGA_PATH, spend_kind=SPEND_KIND)


def intended_duckcreek(*, job_id: str | None) -> dict | None:
    jid = normalize_job_id(job_id)
    if not jid:
        return None
    return write(job_id=jid, path=f"/api/issue/{jid}", spend_kind=SPEND_KIND)


def presented_write(
    *,
    job_id: str | None,
    method: str | None,
    path: str | None,
    spend_kind: str | None = None,
) -> dict | None:
    jid = normalize_job_id(job_id)
    meth = (method or "").strip().upper()
    p = normalize_path(path)
    if not jid or not meth or not p:
        return None
    return write(job_id=jid, method=meth, path=p, spend_kind=spend_kind)


def fingerprints_match(issued: str | None, presented: str | None) -> bool:
    a = (issued or "").strip().lower()
    b = (presented or "").strip().lower()
    return bool(a) and a == b
```

```
def spec(public_url: str) -> dict:
    base = (public_url or "").rstrip("/")
    return {
        "spec": SPEC,
        "name": "Spend protocol",
        "what": "The only scanner that can print a YES on this irreversible spend.",
        "not": [
            "a hop receipt",
            "a second source for the same YES",
            "authorization for bind-and-issue or issue",
        ],
        "married_write": {
            "method": METHOD,
            "path": PATH_TEMPLATE,
            "spend_kind": SPEND_KIND,
        },
        "fingerprint": {
            "alg": "sha256",
            "over": ["method", "path", "job_id", "spend_kind"],
            "canonical_json": "sort_keys, separators=(',', ':')",
        },
        "ticket": {
            "binds_to": "spend_fingerprint",
            "single_use": True,
            "stale_hop_cannot_spend": True,
        },
        "redeem": {
            "path": f"{base}/v1/pas/bind-ticket/redeem",
            "required": ["ticket_id", "token", "job_id", "method", "path", "now"],
            "optional": ["spend_fingerprint", "license_id", "counterpart_id", "counterpart_kind",
  "counterpart_path", "counterpart_method"],
            "now": "UTC. Missing or skewed now is radiation_abort; the ticket is not consumed.",
            "mismatch": REASON_MISMATCH,
            "missing_write": REASON_REQUIRED,
            "not_in_protocol": REASON_NOT_IN_PROTOCOL,
            "fail_closed": True,
            "recompute": "Hash the write being forwarded, not the fingerprint copied off the ticke
  t.",
        },
        "command_radiation": f"{base}/.well-known/command-radiation.json",
        "license_fuse": f"{base}/.well-known/license-fuse.json",
        "restraint": f"{base}/.well-known/restraint.json",
        "refused_writes": [
            "POST /job/v1/jobs/{job_id}/bind-and-issue",
            "POST /policy/v1/policies/{policy_id}/issue",
        ],
        "implementor": f"{base}/listings/cloudflare-worker-bind.js",
        "commit_auth": f"{base}/.well-known/commit-auth.json",
        "their_production": False,
        "page": f"{base}/scanner",
    }
```

ticket.py

```
"""Bind ticket  -  commit-time authorization for irreversible spend.

Ed25519 signs a receipt after the hop. That is ink. This is the capability:
```

```
  A LIVE hop is not a bind grant. The grant is a short-lived, single-use,
  job-bound ticket. After not_after, the hop is a museum.

Lineage (public, not invented here):
- Temporary Authority, Permanent Effects / CommitGuard (2026)  -  witness must
  still be fresh, causally prior, bound to the same target at commit time
- CapLease (2026)  -  Issued -> redeemed once; semantic replay is not a new grant
- MA-Commit non-decomposability (2026)  -  separating validation from execution
  violates safety. Hop-then-bind-later is the decomposition.

SPEED: default TTL is 15 seconds. Cached LIVE cannot spend.
"""
from __future__ import annotations

import hashlib
import json
import os
import secrets
import uuid
from datetime import datetime, timedelta, timezone
from typing import Any

try:
    from gate import receipt as receipt_mod
except ImportError:
    import receipt as receipt_mod

try:
    from gate import db
except ImportError:
    import db

try:
    from gate import spend_protocol
except ImportError:
    import spend_protocol

try:
    from gate import command_radiation
except ImportError:
    import command_radiation

try:
    from gate import counterpart as counterpart_mod
except ImportError:
    import counterpart as counterpart_mod

try:
    from gate import license_fuse as license_fuse_mod
except ImportError:
    import license_fuse as license_fuse_mod

SPEC = "gate-bind-ticket-v1"
DEFAULT_TTL = 15
```

```
def ttl_seconds() -> int:
    try:
        n = int(os.getenv("GATE_BIND_TICKET_TTL", str(DEFAULT_TTL)))
    except ValueError:
        n = DEFAULT_TTL
    return max(1, min(n, 300))


def _canonical(obj: Any) -> str:
    return json.dumps(obj, sort_keys=True, separators=(",", ":"), ensure_ascii=False)


def issue(
    *,
    job_id: str,
    fuse_id: str,
    event_id: str,
    receipt_hash: str | None,
    redeem_url: str,
    spend_write: dict | None = None,
    license_id: str | None = None,
    counterpart: dict | None = None,
) -> dict | None:
    """Mint a bearer ticket. Token is returned once; public receipts never include it.

    Fail closed if there is no spend fingerprint. A job-only ticket is not a print.
    If license_id is present, the parent must be LIVE. Children cannot outlive it.
    """
    jid = (job_id or "").strip()
    fp = spend_protocol.fingerprint(spend_write)
    if not jid or not fp or not isinstance(spend_write, dict):
        return None
    parent = license_fuse_mod.presented(license_id)
    if not parent.get("ok"):
        return None
    lid = parent.get("license_id")
    cp = counterpart if isinstance(counterpart, dict) else {}
    counterpart_fp = (cp.get("fingerprint") or None) if cp.get("fused") else None
    counterpart_write = cp.get("write") if cp.get("fused") else None
    now = datetime.now(timezone.utc)
    ttl = ttl_seconds()
    ticket_id = str(uuid.uuid4())
    token = secrets.token_urlsafe(32)
    write = {
        "method": spend_write.get("method"),
        "path": spend_write.get("path"),
        "job_id": spend_write.get("job_id") or jid,
        "spend_kind": spend_write.get("spend_kind"),
    }
    body = {
        "spec": SPEC,
        "ticket_id": ticket_id,
        "job_id": jid,
        "fuse_id": fuse_id,
        "event_id": event_id,
        "receipt_hash": receipt_hash,
        "not_before": now.isoformat(),
```

```
        "not_after": (now + timedelta(seconds=ttl)).isoformat(),
        "single_use": True,
        "spend": "bind",
        "spend_kind": write["spend_kind"],
        "spend_write": write,
        "spend_fingerprint": fp,
        "license_id": lid,
        "children_cannot_outlive_parent": bool(lid),
        "counterpart_fingerprint": counterpart_fp,
        "counterpart": counterpart_write,
        "ttl_seconds": ttl,
        "stale_hop_cannot_spend": True,
    }
    digest = hashlib.sha256(_canonical(body).encode("utf-8")).hexdigest()
    signature = receipt_mod.sign_receipt_hash(digest)
    db.insert_bind_ticket(
        ticket_id=ticket_id,
        job_id=jid,
        fuse_id=fuse_id,
        event_id=event_id,
        receipt_hash=receipt_hash,
        token_hash=hashlib.sha256(token.encode("utf-8")).hexdigest(),
        not_before=body["not_before"],
        not_after=body["not_after"],
        spend_fingerprint=fp,
        license_id=lid,
        counterpart_fingerprint=counterpart_fp,
    )
    public = {
        **body,
        "signature": signature,
        "redeem": redeem_url,
        "token_present": True,
    }
    bearer = {**public, "token": token}
    return {"bearer": bearer, "public": public}


def public_from_bearer(bearer: dict | None) -> dict | None:
    if not isinstance(bearer, dict):
        return None
    out = {k: v for k, v in bearer.items() if k != "token"}
    return out


def _halt(*, reason: str, ticket_id: str | None = None, job_id: str | None = None, extra: dict | N
  one = None) -> dict:
    out = {
        "ok": False,
        "halt": True,
        "allow_bind": False,
        "radiation_abort": True,
        "reason": reason,
        "spec": SPEC,
        "stale_hop_cannot_spend": True,
    }
    if ticket_id:
```

```
        out["ticket_id"] = ticket_id
    if job_id:
        out["job_id"] = job_id
    if extra:
        out.update(extra)
    return out


def redeem(
    *,
    ticket_id: str,
    token: str,
    job_id: str,
    method: str | None = None,
    path: str | None = None,
    spend_fingerprint: str | None = None,
    spend_kind: str | None = None,
    now: str | None = None,
    license_id: str | None = None,
    counterpart: dict | None = None,
) -> dict:
    """Atomic consume. Fail closed on missing now, skew, stale, mismatch, replay, dead parent, or
  wrong write."""
    server_now = datetime.now(timezone.utc)
    tid = (ticket_id or "").strip()
    tok = (token or "").strip()
    jid = (job_id or "").strip()
    if not tid or not tok or not jid:
        return _halt(reason="ticket_required")
    clock = command_radiation.check_now(now, server=server_now)
    if not clock.get("ok"):
        return _halt(
            reason=clock.get("reason") or command_radiation.REASON_NOW_REQUIRED,
            ticket_id=tid,
            job_id=jid,
            extra={"command_radiation": clock},
        )
    presented = spend_protocol.presented_write(
        job_id=jid,
        method=method,
        path=path,
        spend_kind=spend_kind,
    )
    if presented is None:
        return _halt(
            reason=spend_protocol.REASON_REQUIRED,
            ticket_id=tid,
            job_id=jid,
        )
    presented_fp = spend_protocol.fingerprint(presented)
    claimed = (spend_fingerprint or "").strip()
    if claimed and presented_fp and claimed.lower() != presented_fp.lower():
        return _halt(
            reason=spend_protocol.REASON_MISMATCH,
            ticket_id=tid,
            job_id=jid,
        )
```

```
    row = db.get_bind_ticket(tid)
    issued_lid = ""
    issued_cp = ""
    if row:
        try:
            issued_lid = (row.get("license_id") or "").strip()
        except (AttributeError, KeyError):
            issued_lid = ""
        try:
            issued_cp = (row.get("counterpart_fingerprint") or "").strip()
        except (AttributeError, KeyError):
            issued_cp = ""
    if issued_lid:
        presented_lid = license_fuse_mod.normalize_id(license_id)
        if presented_lid and presented_lid != issued_lid:
            return _halt(
                reason=license_fuse_mod.REASON_MISMATCH,
                ticket_id=tid,
                job_id=jid,
                extra={"license_fuse": license_fuse_mod.snapshot(issued_lid)},
            )
        parent = license_fuse_mod.require_live(issued_lid)
        if not parent.get("ok"):
            return _halt(
                reason=parent.get("reason") or license_fuse_mod.REASON_NOT_LIVE,
                ticket_id=tid,
                job_id=jid,
                extra={"license_fuse": license_fuse_mod.snapshot(issued_lid)},
            )
    cp_parsed = counterpart if isinstance(counterpart, dict) else counterpart_mod.parse({})
    presented_cp = (cp_parsed.get("fingerprint") or "").strip() if cp_parsed.get("ok") else ""
    if issued_cp:
        if not presented_cp or issued_cp.lower() != presented_cp.lower():
            return _halt(
                reason=counterpart_mod.REASON_MISMATCH,
                ticket_id=tid,
                job_id=jid,
            )
    token_hash = hashlib.sha256(tok.encode("utf-8")).hexdigest()
    result = db.consume_bind_ticket(
        ticket_id=tid,
        token_hash=token_hash,
        job_id=jid,
        now=server_now.isoformat(),
        spend_fingerprint=presented_fp,
        counterpart_fingerprint=presented_cp or None,
    )
    if result.get("ok"):
        return {
            "ok": True,
            "halt": False,
            "allow_bind": True,
            "radiation_abort": False,
            "radiated": True,
            "ticket_id": tid,
            "job_id": jid,
            "consumed_at": server_now.isoformat(),
```

```
            "spec": SPEC,
            "single_use": True,
            "spend_fingerprint": presented_fp,
            "spend_write": presented,
            "license_id": issued_lid or None,
            "counterpart_fingerprint": presented_cp or None,
            "command_radiation": clock,
        }
    return _halt(
        reason=result.get("reason") or "ticket_invalid",
        ticket_id=tid,
        job_id=jid,
    )


def stamp(plan: dict, *, ticket_public: dict | None, epoch: dict | None, redeem_url: str) -> dict:
    """Commit-time authorization block on the hop response. Not a museum label."""
    plan["commit_time_authorization"] = {
        "spec": "gate-commit-auth-v1",
        "bind_ticket_required": True,
        "spend_fingerprint_required": True,
        "ttl_seconds": ttl_seconds(),
        "stale_hop_cannot_spend": True,
        "single_use": True,
        "non_decomposable": "A hop without a live ticket is not a bind grant.",
        "married_write": spend_protocol.PATH_TEMPLATE,
        "now_required": True,
        "max_skew_seconds": command_radiation.max_skew_seconds(),
        "license_id_parent": True,
        "children_cannot_outlive_parent": True,
        "counterpart_optional": True,
        "redeem": redeem_url,
        "ticket": ticket_public,
        "epoch": epoch or {"locked": False},
        "lineage": [
            "CommitGuard / commit-time authorization (2026)",
            "CapLease Issued->Consumed (2026)",
            "MA-Commit non-decomposability (2026)",
        ],
    }
    if ticket_public is None and (plan.get("allow_bind") or plan.get("bind_allowed")):
        plan["allow_bind"] = False
        if "bind_allowed" in plan:
            plan["bind_allowed"] = False
        plan["halt"] = True
        plan["commit_time_authorization"]["reason"] = "ticket_unissued"
    return plan


def manifest(public_url: str) -> dict:
    return {
        "spec": SPEC,
        "name": "Bind ticket",
        "greater_than_ed25519": (
            "Signatures prove a hop occurred. Tickets prove the hop is still "
            "allowed to spend, right now, once, for this job and this write."
        ),
```

```
        "ttl_seconds": ttl_seconds(),
        "stale_hop_cannot_spend": True,
        "single_use": True,
        "spend_fingerprint_required": True,
        "married_write": spend_protocol.PATH_TEMPLATE,
        "spend_protocol": f"{public_url}/.well-known/spend-protocol.json",
        "command_radiation": f"{public_url}/.well-known/command-radiation.json",
        "license_fuse": f"{public_url}/.well-known/license-fuse.json",
        "children_cannot_outlive_parent": True,
        "redeem": f"{public_url}/v1/pas/bind-ticket/redeem",
        "demo_redeem": f"{public_url}/demo/pas/bind-ticket/redeem",
        "their_production": False,
    }
```